

PROGRAM 3

IMAGE SEGMENTATION USING U-NET

OBJECTIVE

To perform Image Segmentation using the U-Net Architecture and predict pixel-wise masks for input images.

In this program students will learn:

- ✓ Reading Images Folder
 - ✓ Reading Masks Folder
 - ✓ Image Preprocessing
 - ✓ Building U-Net Architecture
 - ✓ Encoder
 - ✓ Decoder
 - ✓ Skip Connections
 - ✓ Model Training
 - ✓ Mask Prediction
 - ✓ Visualization of Results
-

DATASET FOLDER STRUCTURE

Road_Dataset/

├── Images/

| ├── image1.jpg

| ├── image2.jpg

| └─ ...

|

└─ Masks/

└─ mask1.png

└─ mask2.png

└─ ...

Explanation:

Images Folder

Contains original images.

Masks Folder

Contains segmented ground truth masks.

Each image must have a corresponding mask.

STEP 1 IMPORT LIBRARIES

Purpose:

Import required packages.

Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os

from tensorflow.keras.layers import *
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import load_img
from tensorflow.keras.preprocessing.image import img_to_array
```

```
print("Libraries Imported Successfully")
```

STEP 2 SET IMAGE SIZE

Purpose:

Define fixed image dimensions.

Code:

```
IMG_HEIGHT = 128  
IMG_WIDTH = 128
```

Explanation:

All images and masks will be resized to:

128 × 128

STEP 3 LOAD IMAGES

Purpose:

Read images from folder.

Code:

```
images = []  
  
image_path = "Road_Dataset/Images"  
  
for file in os.listdir(image_path):  
    img = load_img(  
        os.path.join(image_path, file),  
        target_size=(IMG_HEIGHT, IMG_WIDTH)  
    )
```

```
img = img_to_array(img)

img = img/255.0

images.append(img)

print("Images Loaded")
```

Explanation:

load_img()

Reads image.

img_to_array()

Converts image into NumPy array.

img/255

Normalizes pixel values.

STEP 4 LOAD MASKS

Purpose:

Read segmentation masks.

Code:

```
masks = []

mask_path = "Road_Dataset/Masks"

for file in os.listdir(mask_path):

    mask = load_img(
        os.path.join(mask_path, file),
        target_size=(IMG_HEIGHT, IMG_WIDTH),
        color_mode='grayscale'
    )

    mask = img_to_array(mask)
```

```
mask = mask/255.0

masks.append(mask)

print("Masks Loaded")
```

Explanation:

Masks contain:

0 → Background

1 → Object Region

STEP 5 CONVERT TO NUMPY ARRAYS

Purpose:

Prepare data for training.

Code:

```
X = np.array(images)

y = np.array(masks)

print(X.shape)

print(y.shape)
```

Example Output:

```
(100,128,128,3)

(100,128,128,1)
```

STEP 6 CREATE INPUT LAYER

Purpose:

Define input image shape.

Code:

```
inputs = Input(  
    (128,128,3)  
)
```

STEP 7 BUILD ENCODER

Purpose:

Extract image features.

Code:

```
c1 = Conv2D(  
    32,  
    (3,3),  
    activation='relu',  
    padding='same'  
) (inputs)  
  
c1 = Conv2D(  
    32,  
    (3,3),  
    activation='relu',  
    padding='same'  
) (c1)  
  
p1 = MaxPooling2D()(c1)
```

Explanation:

Conv2D

Learns image features.

MaxPooling

Reduces dimensions.

STEP 8 BUILD SECOND ENCODER BLOCK

Code:

```
c2 = Conv2D(
    64,
    (3,3),
    activation='relu',
    padding='same'
)(p1)

c2 = Conv2D(
    64,
    (3,3),
    activation='relu',
    padding='same'
)(c2)

p2 = MaxPooling2D()(c2)
```

STEP 9 BOTTLENECK LAYER

Purpose:

Deep feature extraction.

Code:

```
b1 = Conv2D(
    128,
    (3,3),
    activation='relu',
    padding='same'
)(p2)
```

Explanation:

Bottleneck contains the most important learned features.

STEP 10 BUILD DECODER

Purpose:

Recover image details.

Code:

```
u1 = UpSampling2D()(b1)

u1 = concatenate([u1,c2])

c3 = Conv2D(
    64,
    (3,3),
    activation='relu',
    padding='same'
)(u1)
```

Explanation:

UpSampling

Increases image size.

Concatenate

Creates Skip Connection.

STEP 11 FINAL DECODER BLOCK

Code:

```
u2 = UpSampling2D()(c3)

u2 = concatenate([u2,c1])

c4 = Conv2D(
```



```
    32,  
    (3,3),  
    activation='relu',  
    padding='same'  
) (u2)
```

STEP 12 OUTPUT LAYER

Purpose:

Generate segmentation mask.

Code:

```
outputs = Conv2D(  
    1,  
    (1,1),  
    activation='sigmoid'  
) (c4)
```

Explanation:

Sigmoid

Produces pixel values between:

0 and 1

STEP 13 CREATE MODEL

Code:

```
model = Model(  
    inputs,  
    outputs  
)  
  
model.summary()
```

STEP 14 COMPILE MODEL

Purpose:

Configure training.

Code:

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

STEP 15 TRAIN MODEL

Code:

```
history = model.fit(  
    X,  
    y,  
    epochs=5,  
    batch_size=8  
)
```

STEP 16 PREDICT MASK

Purpose:

Generate segmentation output.

Code:

```
prediction = model.predict(
    X[0:1]
)
```

STEP 17 DISPLAY RESULTS

Purpose:

Visualize segmentation output.

Code:

```
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(X[0])
plt.title("Original")

plt.subplot(1,3,2)
plt.imshow(
    y[0].squeeze(),
    cmap='gray'
)
plt.title("Ground Truth")

plt.subplot(1,3,3)
plt.imshow(
    prediction[0].squeeze(),
    cmap='gray'
)
plt.title("Predicted Mask")

plt.show()
```

EXPECTED OUTPUT

Original Image

↓

Ground Truth Mask



Predicted Mask

Students should clearly observe the segmented object region.

KEY CONCEPTS LEARNED

- ✓ Image Segmentation
 - ✓ Masks
 - ✓ Pixel-Level Classification
 - ✓ U-Net
 - ✓ Encoder
 - ✓ Decoder
 - ✓ Skip Connections
 - ✓ UpSampling
 - ✓ Binary Crossentropy
 - ✓ Mask Prediction
 - ✓ Segmentation Visualization
-

EXPECTED VIVA QUESTIONS

1. What is Image Segmentation?
2. Difference between Classification and Segmentation?
3. Why are Masks required?
4. What is Pixel-Level Classification?

5. What is the role of Encoder?
6. What is the role of Decoder?
7. Why are Skip Connections used?
8. What happens if Skip Connections are removed?
9. Why is Sigmoid used in the Output Layer?
10. Why is Binary Crossentropy used?